

08 - AI Chat and Research

Feature objective

Provide one financial copilot chat that understands relevant structured financial context, can research current information, can trigger deterministic tools/scenarios, and can propose actions without silently mutating financial truth.

Dependencies

- current financial state
- goals
- plan/scenarios
- loans
- backend AI orchestration + SSE
- backend research provider layer

In scope

- AI main tab
- chat thread
- streaming text via SSE
- cancel generation
- suggested starter prompts on empty state
- structured response cards
- research citations/sources
- scenario/goal/plan draft actions
- long-running job state
- retry/error state
- context-aware navigation from other features into AI

Out of scope

- multi-agent visualization
- voice mode
- autonomous plan mutation
- trade execution
- personalized security buy/sell calls without compliance/legal clearance

Route

(tabs)/ai.tsx

AI is a primary tab.

Feature folder

```
features/ai/  
|-- screens/  
|-- components/  
|   |-- MessageList/  
|   |-- Composer/  
|   |-- RecommendationCard/  
|   |-- RiskCard/  
|   |-- ScenarioCard/  
|   |-- ResearchCard/  
|   |-- ActionCard/  
|   |-- SourcesSheet/  
|-- hooks/  
|-- services/
```

```
|-- streaming/  
|-- store/  
|-- utils/  
`-- types.ts
```

Empty chat state

Keep simple:

- short title
- composer
- 3-4 starter prompts

Examples:

- Can I afford a INR 1L laptop?
- Why did my spending increase?
- What should I focus on this month?
- Research SIP options

Suggested prompts disappear after conversation begins.

Streaming architecture

Use one reusable SSE client, not custom stream code per feature.

It must support:

- token/event streaming
- server event types
- abort/cancel
- connection failure
- partial response preservation
- final message commit
- long-running job transition if backend offloads work

Example event families:

```
message.delta  
message.completed  
tool.started  
tool.completed  
research.source  
job.progress  
error
```

Actual event contract belongs to backend/OpenAPI/shared types where possible.

Context assembly

Mobile does not send the entire local database/chat history to the model.

Backend assembles task-relevant structured context.

Example “Can I afford a laptop?” context:

- income
- surplus
- relevant balances
- emergency fund
- EMIs
- active goals
- current plan

Conversation history is supplementary, not canonical financial truth.

Structured response cards

Use cards when the result has actionable/structured meaning.

Recommendation Card

- recommendation
- expected impact
- why it matters
- action buttons

Risk Card

- risk
- severity
- consequence
- suggested mitigation

Scenario Card

- key changed assumption
- baseline vs scenario
- View Scenario action

Research Card

- compared options
- factual metrics
- freshness
- source count
- Sources action

Action Card

- proposed mutation
- explicit Apply/Confirm button

Mutation boundary

AI can:

- explain
- compare
- research
- simulate
- draft
- recommend

AI cannot silently:

- change goal
- change baseline plan
- edit transaction
- edit account balance
- update loan
- change assumptions
- delete data

For a mutation, response creates a draft/proposal and user explicitly applies it.

Deterministic finance rule

AI does not calculate canonical financial results itself.

AI calls backend deterministic tools for:

- affordability
- future cost
- investments/projection
- feasibility
- loan/prepayment
- scenarios
- health score

AI explains returned results.

Research behavior

Research happens through chat in MVP.

Use current external sources only when required.

Research response should expose:

- source name
- source date/freshness
- why the source is relevant
- factual comparison

Source hierarchy should prefer official/regulator/government/issuer/exchange sources over generic web results.

Investment boundary

MVP may:

- explain asset classes
- retrieve factual data about funds/stocks/products
- compare specific products at user request
- simulate allocation effects
- show costs/risk/history/sources

Until legal/compliance review explicitly approves personalized regulated advice, avoid UI wording such as:

- “best fund for you”
- “buy this stock”
- “sell this security”

Prefer:

Here are options worth comparing for this goal, with risks and source-backed facts.

No execution/brokerage actions.

Long-running jobs

If research/plan analysis outlives a normal chat stream:

- backend returns job ID
- UI shows progress card
- user can leave AI tab
- notification/in-app state can indicate completion
- opening completed job restores the final answer

Do not hold a fragile request open for minutes.

Chat persistence

Server stores conversation history needed for continuity.

Important financial outcomes are persisted as structured records, not only chat messages.

Example:

```
chat says "move home goal"
  v
proposal card
  v
user confirms
  v
backend updates goal/creates plan version
```

Error handling

- offline -> explain AI requires internet
- stream interrupted -> preserve partial answer + Retry
- research unavailable -> answer can still explain that fresh research failed
- tool failure -> show tool-specific failure without wiping chat
- auth/session failure -> normal auth recovery

API requirements

Conceptual:

```
GET  /ai/conversations
POST /ai/conversations
GET  /ai/conversations/:id/messages
POST /ai/conversations/:id/messages
GET  /ai/streams/:id or SSE endpoint
POST /ai/jobs/:id/cancel
GET  /ai/jobs/:id
```

Exact structure can be adjusted to backend LangGraph orchestration.

Step-by-step implementation

1. Build empty AI tab + composer.
2. Build message list and persisted conversation load.
3. Add SSE stream client.
4. Add cancel/retry.
5. Add structured card renderer.
6. Add scenario card deep-link.
7. Add explicit proposal/apply actions.
8. Add sources sheet/research card.
9. Add long-running job UI.
10. Add offline/error states.
11. Add cross-feature “Ask AI” prefilled context entry.

Acceptance criteria

- chat streams and can be cancelled
- partial response survives a stream interruption
- AI never mutates financial state without confirmation
- structured scenario/recommendation cards deep-link correctly
- sources show freshness/date when backend returns it
- chat does not require sending full local transaction history from mobile
- financial math displayed in AI is sourced from deterministic backend tool outputs

Tests

- SSE parser/event reducer
- cancel behavior
- stream reconnect/retry state
- structured card rendering
- mutation confirmation path
- offline state
- source sheet
- long-running job completion state